

GEBZE TECHNICAL UNIVERSITY
Department of Computer Engineering

CSE 344 - System Programming

Homework #3 Report

Multi-Process Word Transportation and Concurrent Sorting System

Muhammed Korkmaz

Student ID: 220104004043

April 15, 2026

Contents

1	Introduction	1
2	System Architecture	2
3	Data Structures and Shared Memory Design	2
3.1	Word Structure	2
3.2	CharTask Structure	2
4	Synchronization Strategy	2
5	Word Admission Logic	3
6	Character Transportation and Elevators	3
7	Sorting Algorithm	3
8	Termination Detection and Output Generation	3
9	Challenges and Solutions	4

1 Introduction

The primary objective of this project is to design and implement a multi-process system where multiple independent processes operate concurrently on shared data structures. The system reads words from a text file, decomposes them into characters, transports these characters across different floors via elevators, and reconstructs them concurrently. The main challenge is maintaining data consistency, preventing race conditions, and coordinating multiple active subsystems (carriers, elevators, and sorters) without causing deadlocks or starvation.

2 System Architecture

The system architecture consists of a parent process and several types of child processes, all spawned exclusively by the parent process.

- **Parent Process:** Responsible for parsing command-line arguments, initializing shared memory and synchronization primitives, spawning all child processes, monitoring system completion, and generating the final output file.
- **Word-Carrier Processes:** Bound to their initial arrival floors. They scan the input words using a round-robin approach and admit them into the system if capacity constraints are met.
- **Letter-Carrier Processes:** Mobile processes that pick up character tasks and transport them to their destination floors. They utilize elevators for inter-floor travel.
- **Sorting Processes:** Stationary processes on each floor that reconstruct admitted words character by character according to the original string.
- **Delivery Elevator Process:** Transports characters (carried by letter-carriers) between floors using a direction-based SCAN algorithm.
- **Reposition Elevator Process:** An independent elevator dedicated to moving idle letter-carriers to random floors to prevent starvation.

3 Data Structures and Shared Memory Design

A single shared memory block (`SharedMemory`) is allocated using `mmap` to allow inter-process communication.

3.1 Word Structure

Each word is represented by a `Word` struct, which contains its ID, original string, length, sorting floor, and arrival floor. To manage the state, it holds `claimed`, `admitted`, and `completed` flags. It also contains `sorting_area`, `occupied`, and `fixed` arrays to track the reconstruction progress. A dedicated `sem_t word_mutex` protects these per-word structures.

3.2 CharTask Structure

A global array of `CharTask` structs acts as a task queue. Each task holds metadata: word index, character, original index, source floor, and destination floor. Flags indicating whether the task is claimed and delivered are also included.

4 Synchronization Strategy

POSIX unnamed semaphores are utilized to enforce mutual exclusion and synchronization across processes:

- **admission_mutex:** Ensures atomic access when word-carriers claim words and update floor capacities, preventing race conditions during the all-or-nothing admission phase.
- **task_mutex:** Protects the global character task queue so no two letter-carriers can claim the same character.
- **Elevator Mutexes:** `delivery_mutex` and `reposition_mutex` protect the internal states (passenger counts, queues, and direction) of the respective elevators.
- **word_mutex:** Prevents multiple sorting processes or letter-carriers from concurrently modifying the same word's sorting area or fixed arrays.

5 Word Admission Logic

Word-carrier processes acquire the `admission_mutex` and scan the file's word list in a round-robin manner. For a word to be admitted, an atomic capacity check is performed: both the arrival floor and the targeted sorting floor must have available capacity (`active_words_on_floor < max_words_per_floor`). If successful, the word is marked as admitted, and character tasks are generated.

6 Character Transportation and Elevators

When a letter-carrier claims a task destined for a different floor, it requests the Delivery Elevator. Instead of busy waiting, the carrier increments a waiting counter and blocks on a specific semaphore (`floor_wait_delivery`). The elevator process, following the SCAN algorithm, arrives at the floor, unblocks the carrier up to its capacity limit, and moves to the destination. Upon arrival, it unblocks the carrier via the `ride_wait_delivery` semaphore, allowing the carrier to place the character.

7 Sorting Algorithm

Sorting processes operate strictly on their designated floors. When a sorting process locks a word, it scans the `sorting_area`:

1. If a character is in its correct index and not yet fixed, its `fixed` flag is set to 1.
2. If a character is misplaced and its correct target index is empty, it is moved.
3. If the target index is occupied by another non-fixed character, the two characters are swapped.
4. Fixed positions are strictly protected and never overwritten.

8 Termination Detection and Output Generation

The parent process continuously monitors the system by checking the `completed` flag of all admitted words. Once all words are fully reconstructed, the parent sets a `terminate_flag`,

allowing all child processes to exit their loops gracefully without leaving zombies. Finally, the parent sorts the completed words by `sorting_floor` and `word_id` using `qsort` and writes the result to the output file.

9 Challenges and Solutions

During implementation, several concurrency and robustness issues were addressed:

- **Idle Carrier Repositioning (F2 Implementation):** Initially, idle carriers could cause heavy CPU load by busy-waiting. A dedicated Reposition Elevator was implemented with proper semaphore blocking (`floor_wait_reposition`) to ensure idle carriers sleep until the reposition elevator arrives.
- **Task Queue Overflow (D4 Implementation):** To prevent memory corruption from generating too many character tasks at once, a strict `MAX_TASKS` limit check was added. If the limit is reached, task generation safely breaks.
- **Invalid Floor Inputs (B5 Implementation):** A validation check was added during file reading. If a word specifies a `sorting_floor` greater than the system's total floors or less than zero, it is gracefully ignored, preventing out-of-bounds memory accesses.
- **Output File Authenticity (J3 Implementation):** To prove that the concurrent sorting algorithm actually worked, the final output file prints the dynamically reconstructed string from the `sorting_area` rather than faking it by printing the original parsed string.